

目 录

第 1 章 实验概述	1
1.1 实验目的	1
1.2 实验环境	1
第 2 章 实验内容和步骤	2
2.1 用指令调度技术解决流水线中的结构冲突与数据冲突	2
2.1.1 原始程序执行	2
2.1.2 原始程序冲突分析	3
2.1.3 指令调度优化	3
2.1.4 优化后程序执行结果	4
2.2 用延迟分支技术减少分支指令对性能的影响	5
2.2.1 原始程序执行	5
2.2.2 原始程序分支延迟分析	6
2.2.3 延迟分支优化	7
2.2.4 启用延迟分支后程序执行结果	7
第 3 章 实验总结与心得体会	9

第 1 章 实验概述

1.1 实验目的

1. 加深对指令调度技术的理解。
2. 加深对延迟分支技术的理解。
3. 熟练掌握指令调度技术解决流水线中的数据冲突的方法。
4. 进一步理解指令调度技术对 CPU 性能的改进。
5. 进一步理解延迟分支技术对 CPU 性能的改进。

1.2 实验环境

表 1.1 实验环境

配置项	参数
操作系统	Windows 11 24H2
仿真工具	MIPSSim v1.0.0 64-bit

第2章 实验内容和步骤

2.1 用指令调度技术解决流水线中的结构冲突与数据冲突

2.1.1 原始程序执行

1. **载入程序**：点击菜单栏的 文件 → 载入程序，在弹出的文件选择框中找到并选择 schedule.s 文件，点击打开。
2. **关闭定向功能**：点击菜单栏的 配置，取消勾选定向方式。



图 2.1 schedule.s 未开启定向的执行结果统计

3. **执行程序**：点击菜单栏的 执行 → 连续执行，得到程序的执行结果统计图 2.1 和时间周期图图 2.2。
4. 程序共执行了 33 个周期，发生 16 次 RAW 停顿和 1 次自陷停顿。总停顿周期数占周期总数的 55.52%。

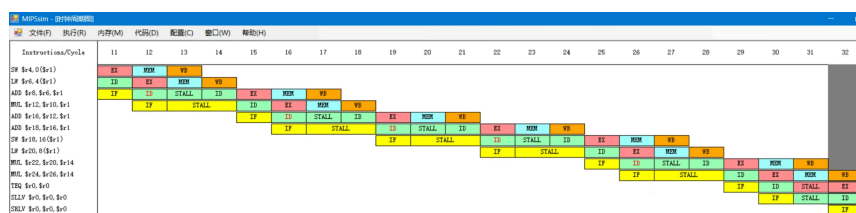


图 2.2 schedule.s 未开启定向的部分时间周期图

2.1.2 原始程序冲突分析

结合程序代码、时钟周期图和停顿统计结果，可以分析出程序中存在数据相关的指令如表 2.1 所示，这些指令导致了 RAW 停顿的发生，严重影响了程序的执行效率。

表 2.1 存在数据相关的指令序列

指令序列 1	指令序列 2
ADDIU \$r1, \$r0, A	LW \$r2, 0(\$r1)
LW \$r2, 0(\$r1)	ADD \$r4, \$r0, \$r2
ADD \$r4, \$r0, \$r2	SW \$r4, 0(\$r1)
LW \$r6, 4(\$r1)	ADD \$r8, \$r6, \$r1
MUL \$r12, \$r10, \$r1	ADD \$r16, \$r12, \$r1
ADD \$r16, \$r12, \$r1	ADD \$r18, \$r16, \$r1
ADD \$r18, \$r16, \$r1	SW \$r18, 16(\$r1)
LW \$r20, 8(\$r1)	MUL \$r22, \$r20, \$r14

2.1.3 指令调度优化

采用调度技术对程序进行指令调度，消除冲突，我优化后的代码 after_schedule.s 如下所示：

1	.text	▲Assembly (x86_64)
2	main:	
3	ADDIU \$r1,\$r0,A	# 初始化基址寄存器
4	MUL \$r24,\$r26,\$r14	# [优化1] 提前执行完全独立指令，用于填充流水线
5	LW \$r2,0(\$r1)	# 载入 A[0]
6	MUL \$r12,\$r10,\$r1	# [优化2] 插入独立计算，延迟 r2 的使用
7	LW \$r6,4(\$r1)	# 载入 A[1]
8	LW \$r20,8(\$r1)	# 载入 A[2] (提前发射，增加并行度)
9	ADD \$r16,\$r12,\$r1	# 使用 r12 (与前面无冲突)

10	ADD	\$r4,\$r0,\$r2	# 使用 r2 (已被延迟, 避免 Load-Use)
11	ADD	\$r8,\$r6,\$r1	# 使用 r6 (已隔离)
12	ADD	\$r18,\$r16,\$r1	# 继续计算链
13	SW	\$r4,0(\$r1)	# 写回 A[0]
14	MUL	\$r22,\$r20,\$r14	# 使用 r20 (已隔离, 避免 Load-Use)
15	SW	\$r18,16(\$r1)	# 写回结果
16	TEQ	\$r0,\$r0	# 程序结束
17			
18	.data		
19	A:		
20	.word	4,6,8	

优化核心包括:

- **消除 Load-Use Hazard**: 在 LW 与其使用之间插入独立指令, 保证间隔 ≥ 1 条指令来消除;
- **提前执行独立指令**: 将完全独立的指令提前执行, 以填充流水线, 提高执行效率;
- **Load 指令前移**: 集中发射, 提前访存, 为后续计算争取时间;
- **交错不同依赖链**: 相互填充延迟槽, 提高并行性;

2.1.4 优化后程序执行结果

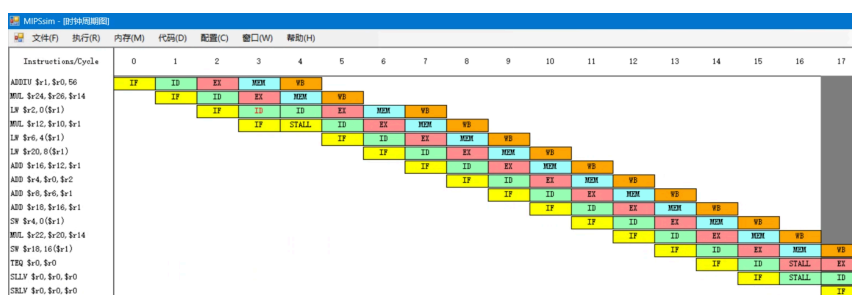


图 2.3 after_schedule.s 的部分时间周期图

执行优化后的程序 after_schedule.s, 得到执行结果统计图 2.4 和时间周期图图 2.3。



图 2.4 after_schedule.s 的执行结果统计

程序共执行了 18 个周期，仅发生 1 次 RAW 停顿，总停顿周期数占周期总数的 11.11%，相比优化前，性能提高了 $33/18 \approx 1.83$ 倍，说明指令调度技术有效地优化了程序性能，显著减少了停顿的发生，从而提升了 CPU 性能。

2.2 用延迟分支技术减少分支指令对性能的影响

2.2.1 原始程序执行

1. **载入程序**：点击菜单栏的 文件 → 载入程序，在弹出的文件选择框中找到并选择 branch.s 文件，点击打开。

2. **关闭延迟分支功能**：点击菜单栏的 配置，取消勾选 延迟槽。

3. **执行程序**：点击菜单栏的 执行 → 连续执行，得到程序的执行结果统计图 2.5 和时间周期图图 2.6。程序共执行了 38 个周期，停顿周期占比为 50%。

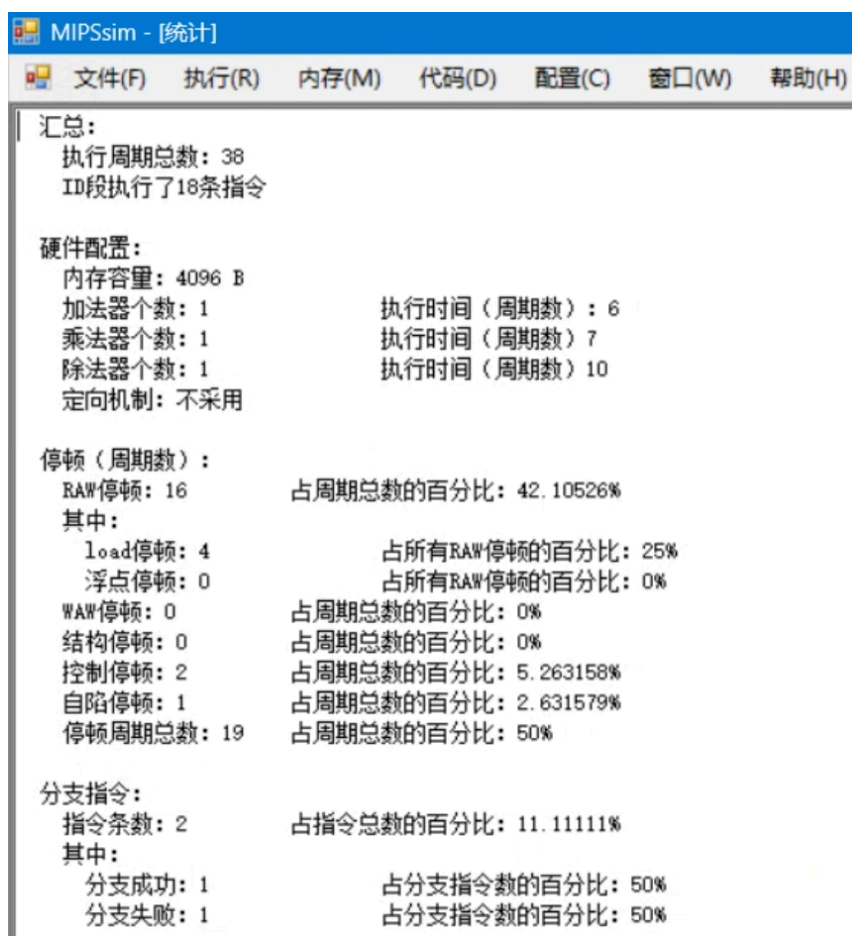


图 2.5 branch.s 未开启延迟分支的执行结果统计

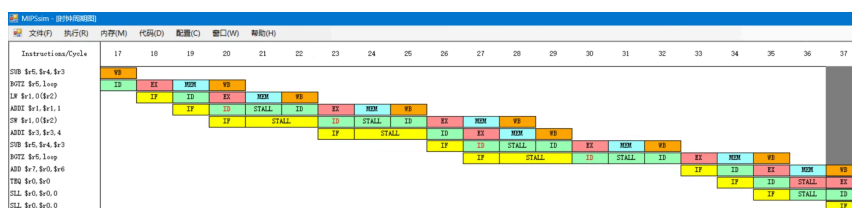


图 2.6 branch.s 未开启延迟分支的部分时间周期图

2.2.2 原始程序分支延迟分析

结合程序代码、时钟周期图和停顿统计结果，程序中存在的分支指令为 BGTZ \$r5, loop。其控制逻辑为：

- $\$r5 = \$r4 - \$r3$
- 当 $\$r5 > 0$ 时跳转到 loop

- 否则顺序执行下一条指令

对于经典 MIPS 流水线，在关闭延迟分支功能时，CPU 在执行 BGTZ 指令时：

- BGTZ 的跳转条件要等 EX/MEM 阶段才能确定
- 但 CPU 已经顺序取了下一条指令 `ADD $r7,$r0,$r6`

由时钟周期图可以得出，共发生了 2 次分支延迟：

- 第一次 BGTZ（跳转发生）：`ADD $r7,$r0,$r6` 进入错误路径 → flush/stall
- 第二次 BGTZ（不跳转）：同样存在判断延迟，但不发生控制流错误

2.2.3 延迟分支优化

假设延迟槽为一个，对 `branch.s` 程序进行指令调度，保存为 `delayed-branch.s`，具体代码如下：

```

1  .text
2  main:
3  ADDI $r2,$r0,1024
4  ADD  $r3,$r0,$r0
5  ADDI $r4,$r0,8
6
7  loop:
8  LW   $r1,0($r2)
9  ADDI $r3,$r3,4      # 提前执行 r3 += 4 (填充LW延迟, 不影响r1语义)
10 ADDI $r1,$r1,1
11 SW   $r1,0($r2)      # 写回内存 (保持 LW→ADD→SW 语义顺序)
12 SUB  $r5,$r4,$r3
13 BGTZ $r5,loop
14 ADD  $r7,$r0,$r6      # branch delay slot
15 TEQ  $r0,$r0

```

2.2.4 启用延迟分支后程序执行结果

执行优化后的程序 `delayed-branch.s`，得到执行结果统计图 2.7 和时间周期图 2.8。

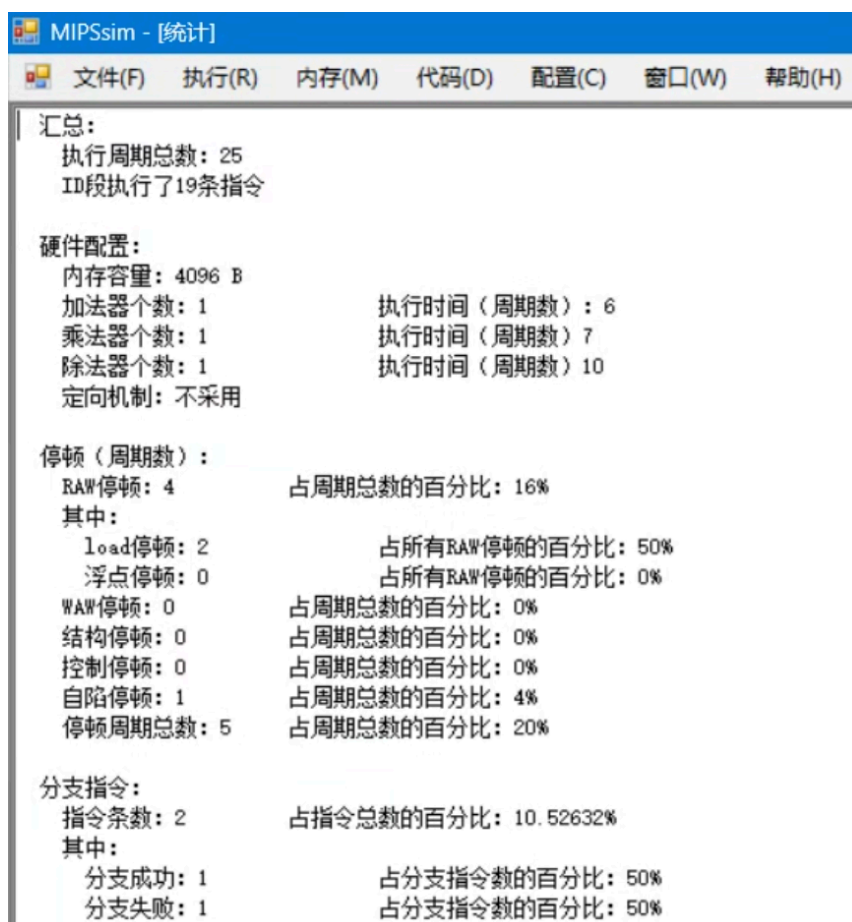


图 2.7 delayed-branch.s 的执行结果统计

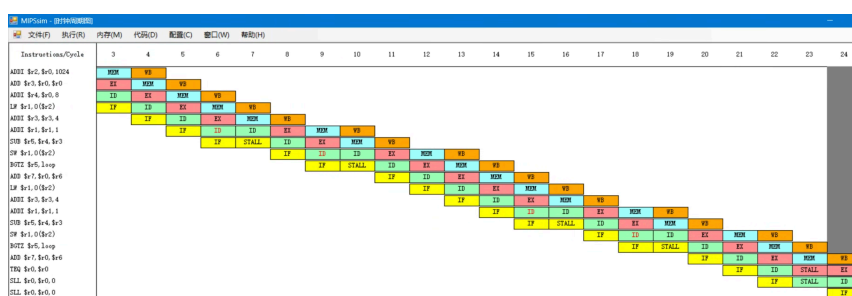


图 2.8 delayed-branch.s 的部分时间周期图

程序共执行了 25 个周期，停顿周期占比为 20%，相比优化前，性能提高了 $38/25 = 1.52$ 倍。可见，延迟分支技术通过让编译器重排指令序列，在分支指令后紧跟一条或几条延迟槽指令，不管分支是否成功，都顺序执行延迟槽中的指令，从而逻辑上延长分支指令的执行时间，减少甚至消除了控制停顿，显著提升了 CPU 性能。

第3章 实验总结与心得体会

通过本次实验，我重温了指令调度与延迟分支的基础知识，并通过实际操作加深了对这些技术的理解。实验中，我首先分析了原始程序中的数据相关性和分支延迟问题，明确了性能瓶颈所在。随后，我通过指令调度技术优化了程序，成功消除了大部分 RAW 停顿，显著提升了性能。接着，我利用延迟分支技术优化了分支指令的执行，进一步减少了控制停顿，提高了程序效率。

综上所述，本次实验不仅让我掌握了指令调度和延迟分支的优化方法，还让我体会到了这些技术在实际 CPU 性能提升中的重要作用。通过分析和优化程序，我深刻理解了指令级并行和控制流优化的原理，这将对我未来的计算机系统设计和性能优化工作产生积极影响。